

Информационный поиск

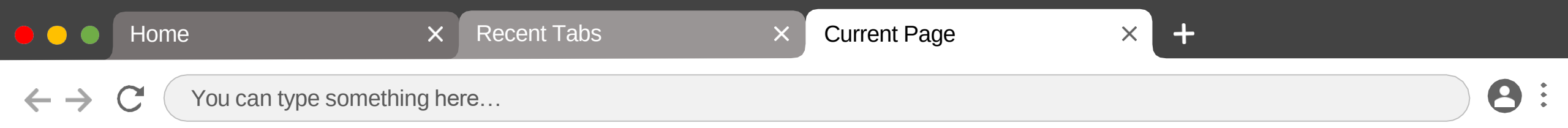
🔍 Деревья поиска 🔊

Ещё раз про индекс

Индекс может храниться в виде матрицы или словаря

```
{
  "буря": [
    "doc_1"
  ],
  "то": [
    "doc_3",
    "doc_4"
  ],
  "как": [
    "doc_3",
    "doc_4"
  ],
  ...
}
```

	буря	мглою	небо	кроет	вихри	снежные	крутя	...
doc_1	1	1	1	1	0	0	0	
doc_2	0	0	0	0	1	1	1	
doc_3	0	0	0	0	0	0	0	
doc_4	0	0	0	0	0	0	0	



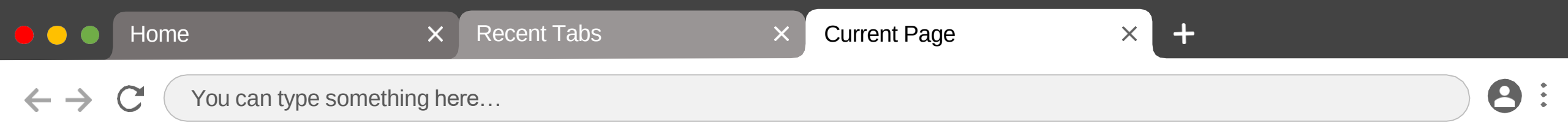
Ещё раз про индекс

Мы много раз обсуждали матричный вид индекса, поговорим теперь про словари.

Нам хочется, чтобы поиск по словарю осуществлялся эффективно. Как этого можно добиться?

Словарь может быть реализован через:

- хеширование
- деревья поиска

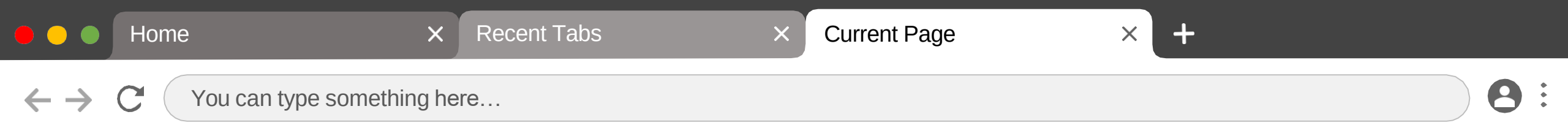


Хэширование

Хэш-функции – функции, преобразующие входные данные в число (битовую строку) установленного размера в соответствии с определённым алгоритмом.

Хэш-функции применяются к ключам словарей при добавлении пары ключ-значение в словарь. При поиске значения по ключу, к ключу снова применяется хэш-функция, и по хэшу (с применением к нему маски) находится ассоциированное с ключом значение. Это обеспечивает константную сложность поиска в среднем.

Минусом этого подхода является существование коллизий – когда значение хэша от разных ключей совпадает.



Деревья поиска

Дерево поиска – древовидная структура данных, в которой узлы упорядочены определённым образом по значению ключей.

Свойство упорядоченности для каждого узла: все ключи в левом поддереве меньше, чем ключ текущего узла, все ключи в правом поддереве больше, чем ключ текущего узла. Каждое поддерево также является деревом поиска.

Двоичное дерево

Самым простым примером дерева поиска является **двоичное**, или **бинарное дерево (BST)**.

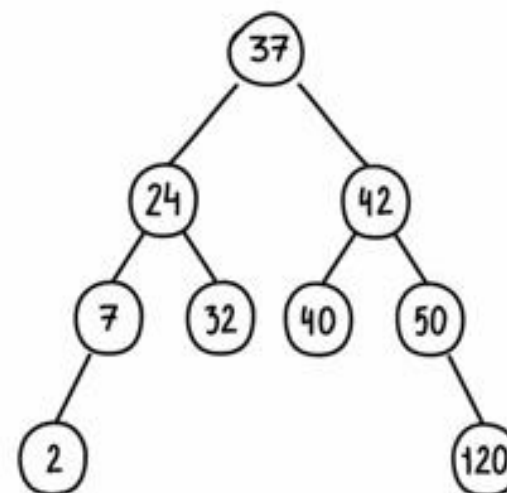
Рассмотрим его структуру.

Почему оно называется двоичным?

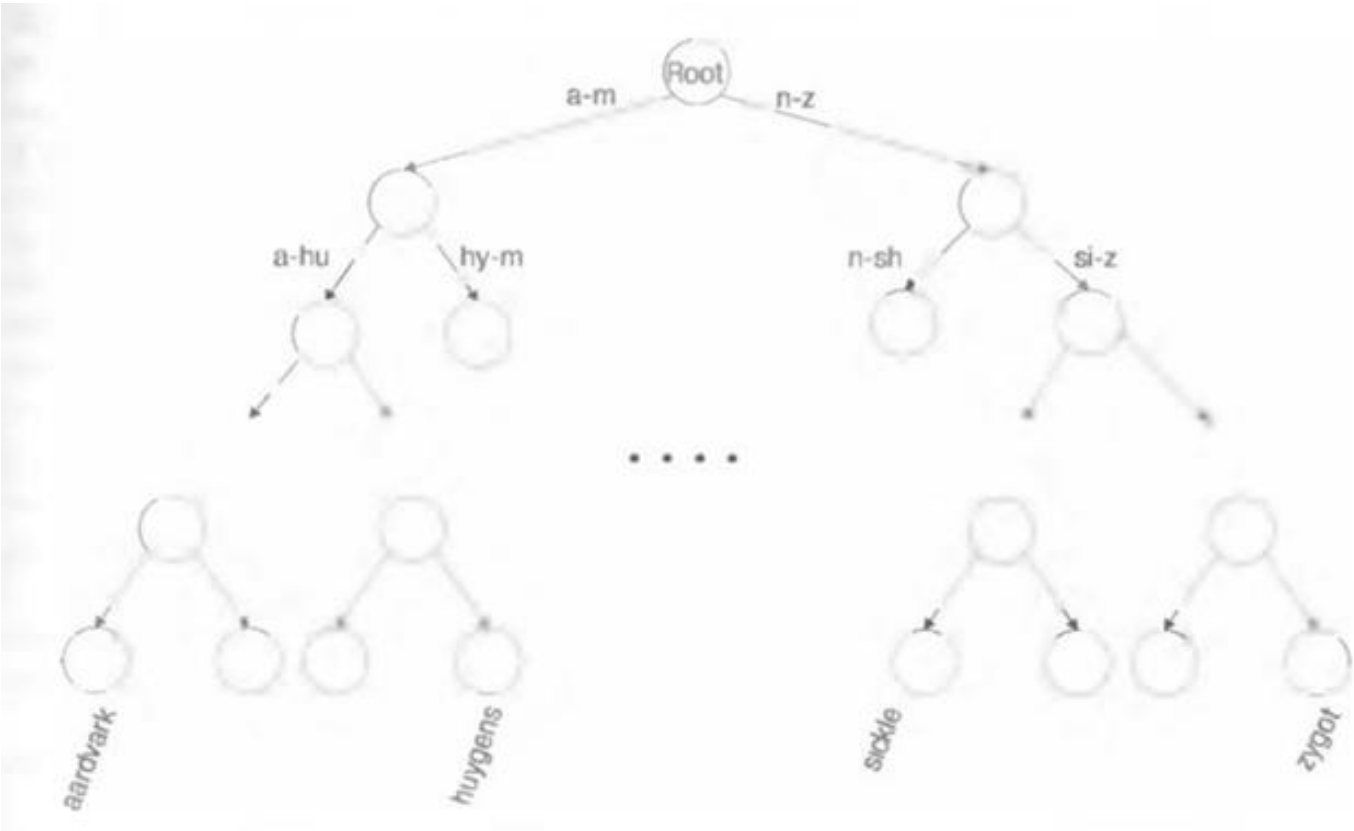
Является ли оно
сбалансированным?
Почему?

Как добавлять в него новые элементы?
Как удалять старые?

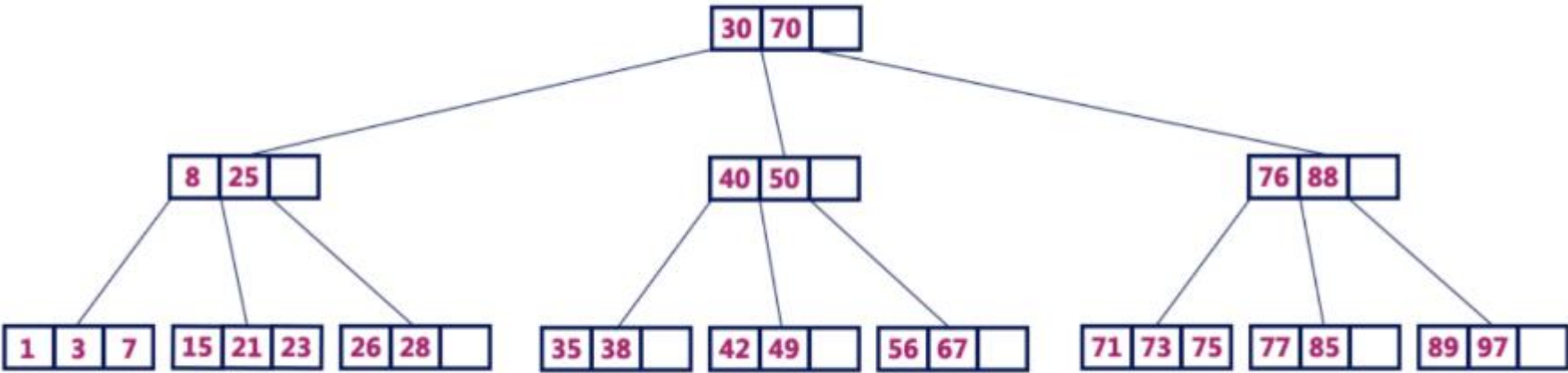
Какова сложность поиска в нём?



Поиск по двоичному дереву индекса



В-деревья

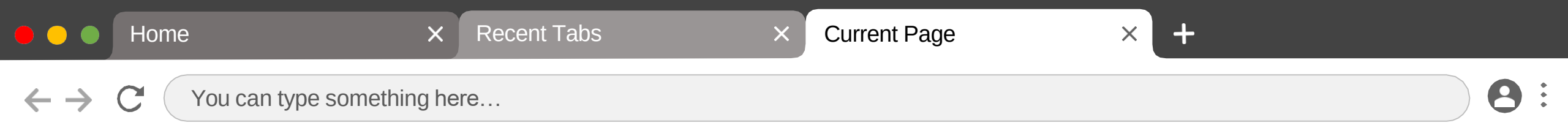


В-дерево 4 порядка

Свойства В-деревьев

- Глубина всех листьев одинакова
- Все узлы, кроме корня должны иметь как минимум $(m/2) - 1$ ключей и максимум $m-1$ ключей
- Все узлы без листьев, кроме корня (т.е. все внутренние узлы), должны иметь минимум $m/2$ потомков
- Если корень – это узел не содержащий листьев, он должен иметь минимум 2 потомка
- Узел без листьев с $n-1$ ключами должен иметь n потомков
- Все ключи в узле должны располагаться в порядке возрастания их значений

[Статья с хорошим разбором В-деревьев](#)



Зачем это нужно?

С ростом базы данных разница всё заметнее.

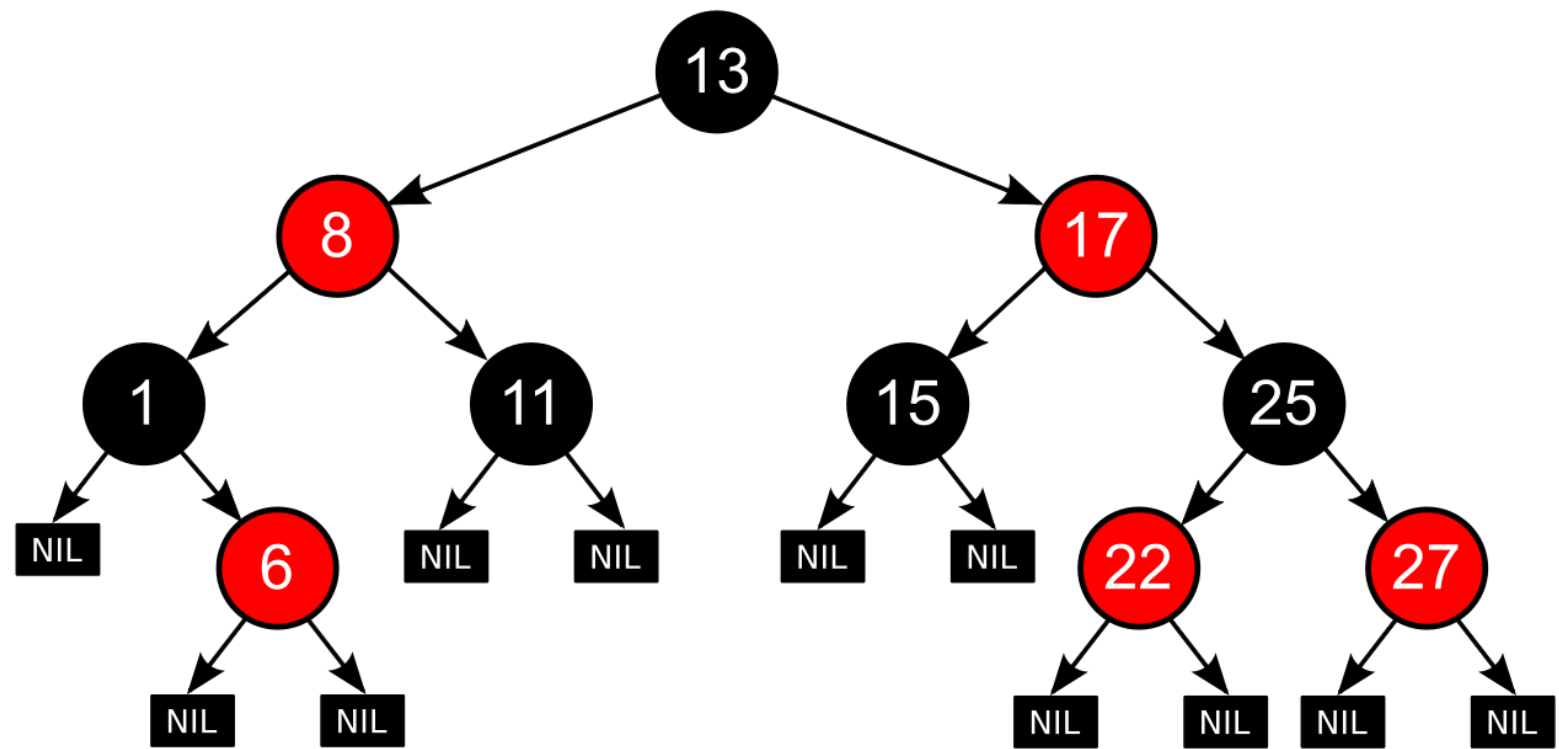
Если нужно хранить миллион значений, то вам понадобится:

- Двоичное дерево поиска высотой **20** уровней
- ИЛИ
- Дерево с узлами из трёх значений высотой **10** уровней

Каждый уровень – это новое обращение к диску, которое стоит «дороже», чем последовательное считывание данных из одного узла.

Подробнее: [тут](#).

Красно-чёрные деревья



Красно-чёрные деревья

Принципы устройства:

1. Каждый узел промаркирован красным или чёрным цветом
2. Корень и конечные узлы (листья) дерева — чёрные
3. У красного узла родительский узел — чёрный
4. Все простые пути из любого узла x до листьев содержат одинаковое количество чёрных узлов
5. Чёрный узел может иметь чёрного родителя.

Всегда считаем именно чёрную глубину (почему этого достаточно?)

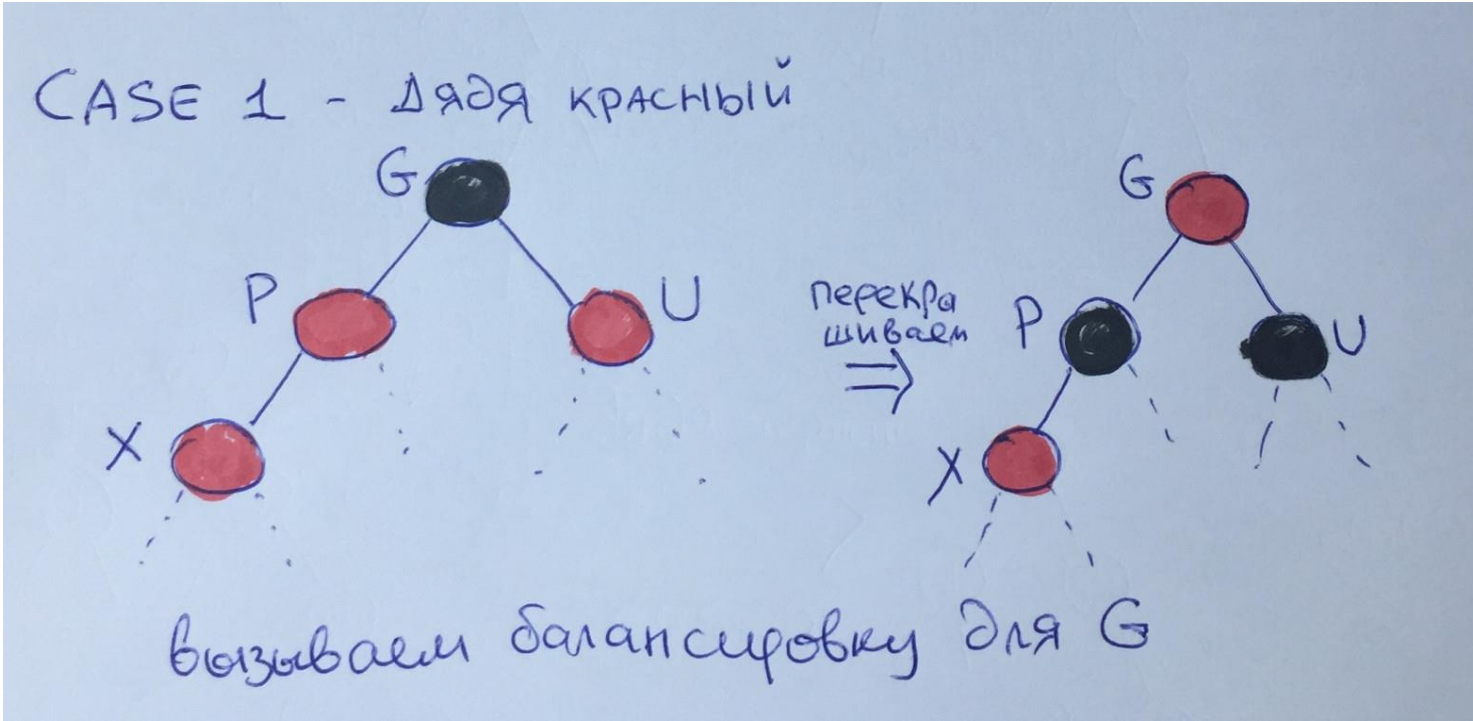
Сложность поиска — $O(\log N)$.

Красно-чёрные деревья: механизмы балансировки

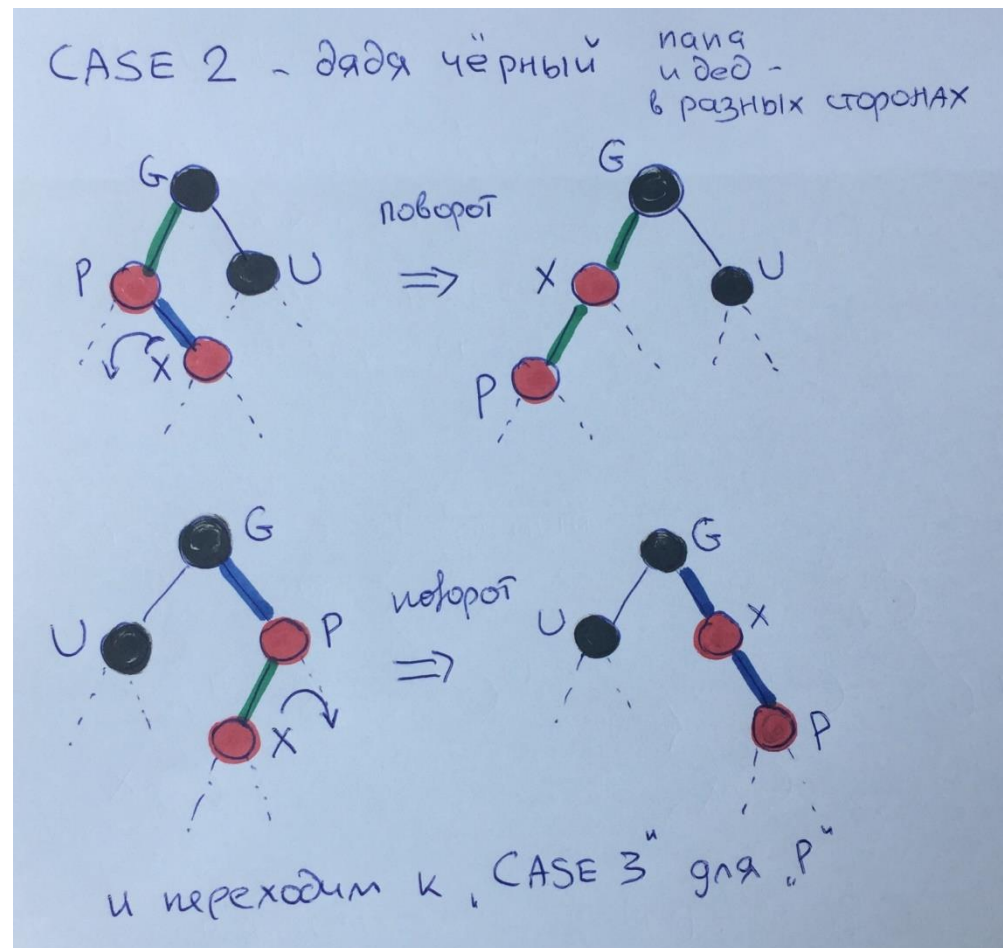
Вставляем всегда красный элемент (если это не корень).
Дальше возможны разные варианты.

[Статья с картинками](#)

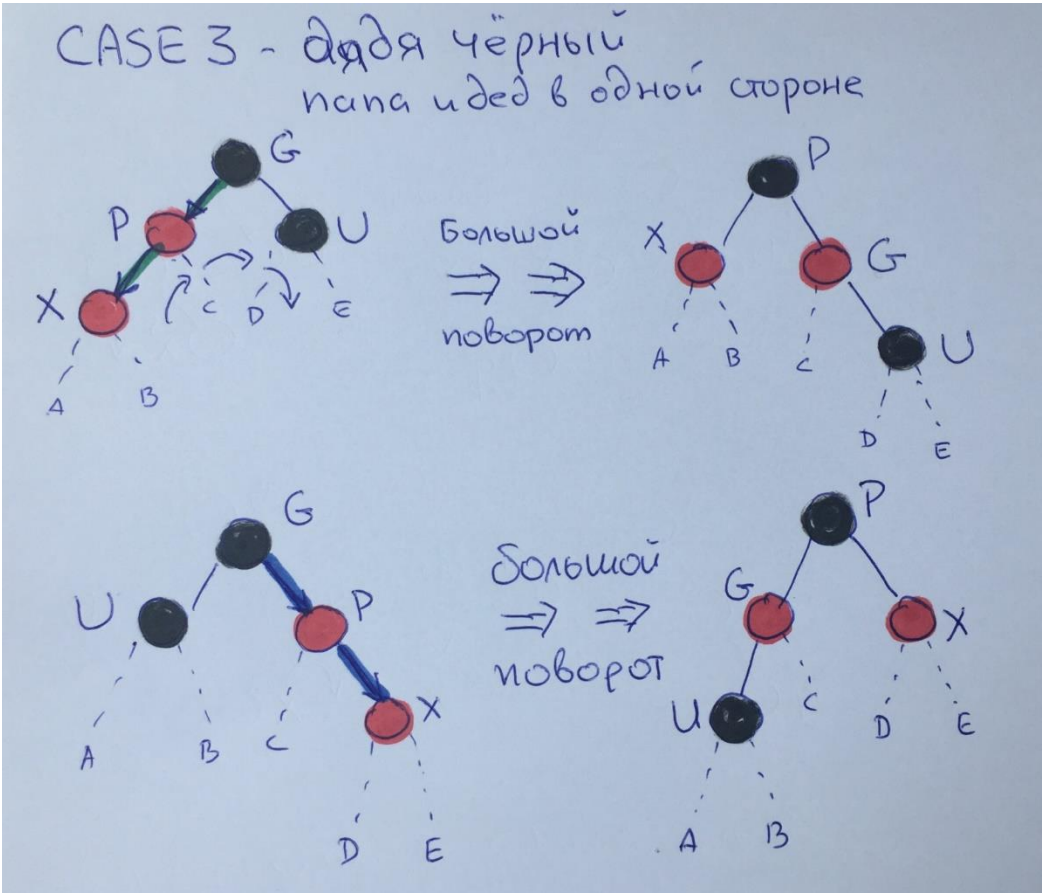
Красно-чёрные деревья: механизмы балансировки

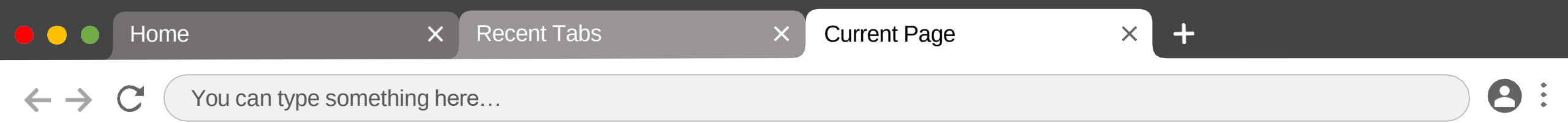


Красно-чёрные деревья: механизмы балансировки



Красно-чёрные деревья: механизмы балансировки





Современные поисковые движки

- ElasticSearch
 - Solr
- (оба на базе Lucene)

Хорошее описание того, как устроено индексирование и поиск в ElasticSearch: раз, два